

Kreacijski paterni u projektu BMDb

Singleton patern

Uloga: Osigurava da određena klasa ima samo jednu instancu tokom izvršavanja aplikacije i obezbeđuje tačku pristupa toj instanci.

Problem u projektu BMDb: Jedan od problema predstavlja otvaranje nove konekcije prema bazi podataka pri svakom korisničkom zahtjevu za prikaz sadržaja, što bi opterećavalo server. Također, centralizovano logovanje grešaka zahtijeva jedinstvenu instancu kako ne bi dolazilo do konflikata pri upisu u fajl.

Primjena u Modelu: BMDbContext / DatabaseConnection : Implementira se kao Singleton sa privatnim konstruktorom i statičkom thread-safe instancom. Svi repozitoriji modela koriste ovaj jedinstveni kontekst za komunikaciju sa bazom.

- LoggerService: Koristi se za centralizovano praćenje rada sistema i registrovanje grešaka prilikom alokacije resursa ili neuspjelih korisničkih prijava.

Prototype patern

Uloga: Omogućava kloniranje postojećih objekata (prototipova) kako bi se kreirali novi, bez direktnog pozivanja konstruktora i ponovnog mapiranja podataka.

Problem u projektu BMDb: Prilikom unosa filmova u bazu, moderator često unosi filmove koji pripadaju istoj franšizi. Ti filmovi dijele 90% istih informacija: isti žanr, istog režisera, istu producentsku kuće. Unositi sve ove podatke potpuno ispočetka za svaki nastavak je gubljenje vremena i podložno je greškama.

Primjena u Modelu: Klasa Film implementira interfejs sa metodom Clone(). Kada moderator želi unijeti novi nastavak filma, on jednostavno izabere prethodni dio koji služi kao prototip (šablon). Sistem otvara formu

sa već popunjenim podacima, a moderator samo promijeni naslov i godinu izlaska i sačuva novi objekat.

Factory Method patern

Uloga: Definiše interfejs ili apstraktnu klasu za kreiranje objekata, ali prepušta izvedenim klasama da odluče koju će konkretnu klasu instancirati.

Problem u projektu BMDb: Glavni medijski sadržaj na platformi se dijeli na filmove i serije. Iako oba entiteta dijele zajednička polja (ID, naslov, godina izdanja, režiser), oni imaju i bitne specifičnosti u modelu podataka: film ima trajanje u minutama, dok serija ima broj sezona i listu epizoda. Direktno instanciranje unutar koda stvara rigidnu strukturu.

Primjena u Modelu: Uvodi se apstraktna klasa MediaContent iz koje se izvode Film i Serija. Kreira se MediaFactory koja na osnovu prosljeđenog parametra dinamički generiše odgovarajući tip objekta. Ovo omogućava da se u budućnosti bez izmjene postojeće logike dodaju novi tipovi sadržaja.

Abstract Factory patern

Uloga: Omogućava kreiranje porodica povezanih ili zavisnih objekata bez specificiranja njihovih konkretnih klasa.

Problem u projektu BMDb: Korisnički nalozi u BMDb sistemu podijeljeni su na goste, registrovane korisnike, verifikovane recenzente, moderatore i administratore. Svaki od ovih tipova naloga zahtijeva kreiranje čitave porodice povezanih objekata u sistemu: specifičan podsistem za obradu recenzija (verifikovani recenzenti ostavljaju detaljne recenzije).

Primjena u Modelu: Uvodi se apstraktna fabrika UserComponentFactory sa metodama poput CreateReviewHandler(). Iz nje se, na primjer izvode StandardUserComponentFactory i PremiumUserComponentFactory. Kada se korisnik prijavi, sistem automatski aktivira odgovarajuću fabriku koja garantuje konzistentno kreiranje isključivo onih komponenti koje pripadaju toj porodici korisničkog nivoa.

Builder patern

Uloga: Služi za izgradnju kompleksnih objekata korak po korak, posebno kada objekat sadrži veliki broj opcionih atributa i parametara.

Problem u projektu BMDb: Napredna pretraga filmova omogućava korisniku da bira veliki broj različitih filtera (žanr, godina, ocjena). Najčešće korisnik izabere samo jedan ili dva filtera (npr. samo žanr „Komediya“). Da nema Builder paterna, sistem bi morao imati ogroman i nepregledan konstruktor u kojem bismo za sve ostale filtere koje korisnik nije izabrao morali ručno pisati null ili prazna polja, što kod čini nečitljivim i podložnim greškama.

Primjena u Modelu: Uvodi se klasa MovieSearchFilterBuilder. Ona omogućava da se filter gradi postepeno, pozivanjem metoda samo za one kriterije koje je korisnik stvarno izabrao. Na kraju, metoda .Build() spaja te izabrane dijelove u jedan spreman i čist objekat.